

Platformy Programistyczne

FinTrack

System zarządzania finansami osobistymi

Kamil Klepusewicz 280118

12.06.2026r.

Spis treści

1. Opis i cel projektu	2
2. Zakres funkcjonalny aplikacji.....	2
3. Architektura systemu	2
4. Warstwa backendowa	3
5. Warstwa frontendowa	3
6. Baza danych i Entity Framework Core	4
7. Logowanie i bezpieczeństwo	4
8. Integracja z API NBP	4
9. Testy.....	5
10. Dokumentacja	5
11. Konteneryzacja i uruchomienie aplikacji	6
12. Podsumowanie	6

1. Opis i cel projektu

Celem projektu było stworzenie aplikacji webowej **FinTrack**, służącej do zarządzania finansami osobistymi. System umożliwia użytkownikowi rejestrowanie kont finansowych, dodawanie przychodów i wydatków, przypisywanie transakcji do kategorii oraz bieżące kontrolowanie salda.

Aplikacja została przygotowana jako system klient-serwer, w którym frontend odpowiada za interfejs użytkownika, a backend za logikę biznesową, komunikację z bazą danych oraz integrację z zewnętrznym API. Jednym z ważniejszych elementów projektu jest obsługa transakcji walutowych. W przypadku operacji wykonanych w walucie innej niż PLN aplikacja pobiera kurs z API Narodowego Banku Polskiego i przelicza wartość transakcji na złotówki.

Projekt miał na celu połączenie kilku elementów omawianych w ramach przedmiotu: aplikacji webowej, bazy danych, komunikacji przez API, logowania użytkownika, obsługi formularzy, walidacji danych oraz konteneryzacji środowiska.

2. Zakres funkcjonalny aplikacji

W ramach projektu zrealizowano następujące funkcjonalności:

- rejestrację i logowanie użytkownika,
- autoryzację z użyciem tokenów JWT,
- dodawanie, edycję i usuwanie kont finansowych,
- dodawanie transakcji przychodowych i wydatkowych,
- przypisywanie transakcji do kategorii,
- automatyczne aktualizowanie salda konta po dodaniu transakcji,
- obsługę transakcji w walutach obcych,
- pobieranie kursów walut z API NBP,
- przeliczanie wartości transakcji walutowych na PLN,
- prezentację danych w tabelach i kartach podsumowujących,
- podstawową walidację danych wejściowych,
- obsługę błędów po stronie backendu,
- automatyczną dokumentację API w Swaggerze,
- dokumentację techniczną wygenerowaną przy użyciu DocFX,
- testy integracyjne backendu,
- uruchamianie aplikacji w kontenerach Docker.

3. Architektura systemu

Aplikacja została zbudowana w architekturze klient-serwer. Poszczególne części systemu są od siebie rozdzielone, dzięki czemu frontend, backend oraz baza danych mogą być rozwijane i uruchamiane niezależnie.

System składa się z trzech głównych warstw:

1. **Frontend** - aplikacja React odpowiedzialna za interfejs użytkownika.
2. **Backend** - API napisane w ASP.NET Core, obsługujące logikę aplikacji.
3. **Baza danych** - SQLite, wykorzystywana do trwałego zapisu danych.

Komunikacja między frontendem a backendem odbywa się za pomocą zapytań HTTP, a dane są przesyłane w formacie JSON.

4. Warstwa backendowa

Backend aplikacji został wykonany w technologii **ASP.NET Core Web API**. Odpowiada on za obsługę żądań przychodzących z frontendu, walidację danych, komunikację z bazą danych oraz integrację z API NBP.

W backendzie zastosowano podział na kontrolery odpowiedzialne za konkretne części systemu:

- AuthController - rejestracja, logowanie i obsługa tokenów JWT,
- AccountsController - operacje na kontach użytkownika,
- TransactionsController - dodawanie i pobieranie transakcji,
- CategoriesController - obsługa kategorii transakcji.

Dzięki takiemu podziałowi kod jest bardziej czytelny, a odpowiedzialności poszczególnych modułów są jasno rozdzielone.

Do komunikacji z zewnętrznym API wykorzystano HttpClient. Zapytania do API NBP są wykonywane asynchronicznie, dzięki czemu aplikacja nie blokuje działania serwera podczas oczekiwania na odpowiedź z zewnętrznego serwisu.

5. Warstwa frontendowa

Frontend został przygotowany jako aplikacja typu **Single Page Application** z wykorzystaniem biblioteki **React** oraz środowiska **Vite**. Interfejs użytkownika został zbudowany z komponentów, co ułatwia zarządzanie widokami oraz ponowne wykorzystywanie elementów UI.

Do stylowania użyto **React-Bootstrap**, dzięki czemu aplikacja ma responsywny układ i może być wygodnie używana zarówno na ekranach komputerów, jak i urządzeń mobilnych.

Frontend komunikuje się z backendem przez zapytania HTTP. Po wykonaniu operacji, takich jak dodanie konta lub transakcji, widok jest aktualizowany bez konieczności przeładowywania całej strony.

Główne elementy interfejsu użytkownika to:

- formularze logowania i rejestracji,
- widok kont użytkownika,
- formularz dodawania transakcji,
- tabela transakcji,
- karty z podsumowaniem salda,

- widoki kategorii i podstawowych danych finansowych.

6. Baza danych i Entity Framework Core

Do przechowywania danych wykorzystano relacyjną bazę **SQLite**. Jest to lekkie rozwiązanie, które dobrze sprawdza się w projekcie studenckim, ponieważ nie wymaga instalowania osobnego serwera bazodanowego.

Dostęp do danych został zrealizowany z użyciem **Entity Framework Core**. W projekcie zastosowano podejście Code First, w którym struktura bazy danych wynika z klas modeli zdefiniowanych w kodzie aplikacji.

W bazie przechowywane są między innymi dane dotyczące:

- użytkowników,
- kont finansowych,
- kategorii,
- transakcji,
- kursów walut pobieranych z API NBP.

Między tabelami występują relacje, np. użytkownik może posiadać wiele kont, a konto może mieć wiele przypisanych transakcji.

7. Logowanie i bezpieczeństwo

W aplikacji zaimplementowano mechanizm logowania oparty na tokenach **JWT**. Po poprawnym zalogowaniu backend generuje token, który następnie jest przechowywany po stronie klienta i dołączany do kolejnych zapytań kierowanych do API.

Token jest przesyłany w nagłówku:

Authorization: Bearer <token>

Dzięki temu backend może sprawdzić, czy użytkownik jest zalogowany i czy ma prawo dostępu do określonych zasobów. Pozwala to również odseparować dane poszczególnych użytkowników, tak aby każdy użytkownik widział wyłącznie swoje konta i transakcje.

8. Integracja z API NBP

Jednym z kluczowych elementów projektu jest integracja z publicznym API Narodowego Banku Polskiego. Aplikacja korzysta z API NBP w celu pobierania kursów walut.

W przypadku dodawania transakcji w walucie innej niż PLN backend wysyła zapytanie do endpointu NBP, np.:

api.nbp.pl/api/exchangerates/rates/a/{currency}/

Po otrzymaniu odpowiedzi w formacie JSON aplikacja odczytuje średni kurs waluty i wykorzystuje go do przeliczenia wartości transakcji na PLN.

Przykład działania:

- Kwota transakcji: 100 EUR
- Kurs EUR: 4,30 PLN
- Wartość po przeliczeniu: 430 PLN

Wartość przeliczona na PLN jest następnie zapisywana w bazie danych razem z oryginalną kwotą i walutą transakcji. Dzięki temu użytkownik może prowadzić finanse w różnych walutach, a aplikacja nadal pozwala analizować je w jednej walucie bazowej.

Obsłużono również sytuacje błędne, takie jak brak odpowiedzi z API NBP lub niepoprawny kod waluty. W takich przypadkach aplikacja nie zapisuje niespójnych danych i zwraca informację o błędzie.

9. Testy

W celu sprawdzenia poprawności działania najważniejszych mechanizmów przygotowano testy integracyjne backendu z wykorzystaniem **NUnit** oraz bazy danych działającej w pamięci.

Testy obejmują między innymi:

- poprawne dodawanie transakcji,
- aktualizację salda konta po dodaniu wydatku,
- obsługę transakcji walutowej,
- działanie logiki biznesowej bez zapisu do rzeczywistej bazy danych.

Zastosowanie bazy InMemory pozwala uruchamiać testy szybko i bez ryzyka modyfikacji właściwych danych aplikacji. Każde uruchomienie testów korzysta z osobnej, tymczasowej bazy danych tworzonej w pamięci RAM.

Testy można uruchomić komendą:

```
dotnet test
```

10. Dokumentacja

W projekcie przygotowano dwa rodzaje dokumentacji.

Pierwszym z nich jest dokumentacja API dostępna przez **Swagger UI**. Pozwala ona sprawdzić dostępne endpointy, strukturę zapytań i odpowiedzi oraz testować wybrane metody backendu bez konieczności korzystania z frontendu.

Drugim elementem jest dokumentacja techniczna wygenerowana za pomocą **DocFX**. Na podstawie komentarzy XML oraz struktury kodu wygenerowano statyczną stronę HTML opisującą klasy, modele i metody użyte w projekcie.

Dokumentacja DocFX znajduje się w katalogu:

```
backend/_site/
```

Można ją otworzyć przez plik index.html lub uruchomić lokalnie przy pomocy komendy:

```
docfx docfx.json --serve
```

11. Konteneryzacja i uruchomienie aplikacji

Aplikacja została skonteneryzowana z użyciem **Docker Compose**. Dzięki temu można uruchomić frontend i backend jedną komendą, bez ręcznego konfigurowania środowiska.

Aby uruchomić projekt, należy w katalogu głównym aplikacji wykonać polecenie:

```
docker compose up -d --build
```

Po uruchomieniu dostępne są następujące adresy:

- Frontend: <http://localhost:5173>
- Backend API: <http://localhost:5074>
- Swagger UI: <http://localhost:5074/swagger>

Konteneryzacja ułatwia testowanie projektu na różnych komputerach i ogranicza problemy wynikające z różnic w konfiguracji środowiska.

12. Podsumowanie

Projekt FinTrack został zrealizowany jako aplikacja webowa do zarządzania finansami osobistymi. W ramach pracy przygotowano zarówno frontend w React, jak i backend w ASP.NET Core Web API. Aplikacja umożliwia użytkownikowi tworzenie kont finansowych, dodawanie transakcji, kategoryzowanie wydatków oraz obsługę transakcji walutowych.

Najważniejszym elementem wyróżniającym projekt jest integracja z API Narodowego Banku Polskiego. Dzięki niej aplikacja może automatycznie pobierać kursy walut i przeliczać transakcje na PLN. Pozwala to lepiej odwzorować rzeczywiste działanie aplikacji finansowej i pokazuje praktyczne użycie zewnętrznego API.

Projekt spełnia założenia przedmiotu, ponieważ wykorzystuje bazę danych, ORM, komunikację HTTP, format JSON, logowanie, walidację, obsługę wyjątków, testy, dokumentację oraz konteneryzację. Dodatkowo aplikacja została przygotowana w sposób umożliwiający dalszą rozbudowę, np. o raporty, budżety miesięczne, cele oszczędnościowe lub bardziej zaawansowaną analizę wydatków.